

Mongoose

Mongoose

- **Mongoose** is an **Object Data Modeling (ODM) library** for **MongoDB** and **Node.js**.
- It provides a **straightforward and schema-based** way to interact with MongoDB databases in Node.js applications.

Features

- **Schema-based Modeling:** Define how your documents should look using schemas.
- **Data Validation:** Enforces data types and rules (e.g., required fields).
- **Middleware Support:** Add logic before or after events (e.g., before saving a document).
- **Built-in Type Casting:** Automatically converts data types (e.g., strings to dates).
- **Query Building:** Simplifies complex MongoDB queries.
- **Easy CRUD operations**

Why Use Mongoose?

- While MongoDB is schema-less, Mongoose gives structure to your data.
- This is especially useful for building reliable and maintainable applications.

Example Use Case

- If you're building a student record system, you can define a Student schema with name, regNumber, and age, and Mongoose will handle **how the data is stored, validated, and retrieved from MongoDB.**

Connecting to MongoDB with Node.js (Without Mongoose)

Install mongodb driver:

- `npm init -y`
- `npm install mongodb`

Connecting to MongoDB with Node.js (Without Mongoose)

connectMongo.js

```
const { MongoClient } = require('mongodb');  
// MongoDB local connection  
const uri = 'mongodb://localhost:27017';  
const client = new MongoClient(uri);
```

```
async function run() {  
  try {  
    await client.connect();  
    console.log('Connected to MongoDB');  
  
    const db = client.db('testDB');  
    const collection = db.collection('students');
```

```
    const result = await  
collection.insertOne({ name: 'John', age: 21 });  
    console.log('Inserted:', result.insertedId);  
  } catch (err) {  
    console.error(err);  
  }  
  finally {  
    await client.close();  
  }  
}
```

```
run();
```

- **To run: node connectMongo.js**

Connecting to MongoDB using Mongoose

To install mongoose:

Install it:

npm install mongoose

mongooseConnect.js

```
const mongoose = require('mongoose');
```

```
// Connect to local MongoDB
```

```
mongoose.connect('mongodb://  
localhost:27017/studentDB', {  
  useNewUrlParser: true,
```

```
  useUnifiedTopology: true,  
});
```

```
const db = mongoose.connection;
```

```
db.on('error', console.error.bind(console,  
'Connection error:'));  
db.once('open', () => {  
  console.log('Connected to MongoDB via  
Mongoose');  
});
```

Defining Mongoose Schemas

Schema: Defines the structure of a document.

Model: A constructor based on a schema to create and interact with documents.

studentModel.js

```
const mongoose = require('mongoose');
```

```
const studentSchema = new  
mongoose.Schema({
```

```
  name: {
```

```
    type: String,
```

```
    required: true,
```

```
  },
```

```
  registerNumber: {
```

```
    type: String,
```

```
    required: true,
```

```
    unique: true,
```

```
  },
```

```
  age: Number,
```

```
});
```

```
const Student =
```

```
mongoose.model('Student',  
studentSchema);
```

```
module.exports = Student;
```

CRUD Operations using Mongoose

app1.js (Main app)

```
const mongoose =  
require('mongoose');
```

```
const Student =  
require('./studentModel');
```

// Connect to MongoDB

```
mongoose.connect('mongodb://  
localhost:27017/studentDB', {  
  useNewUrlParser: true,  
  useUnifiedTopology: true,  
});
```

// Create (Insert)

```
async function createStudent() {  
  const student = new Student({  
    name: 'Alice',  
    registerNumber: 'REG101',  
    age: 22,  
  });  
  await student.save();  
  console.log('Student Created:',  
student);  
}
```


CRUD Operations using Mongoose

contd...

// Read (Find)

```
async function getStudents() {  
  const students = await  
  Student.find();  
  
  console.log('All Students:',  
students);  
}
```

// Update

```
async function  
updateStudent(regNo) {
```

```
  const student = await  
  Student.findOneAndUpdate(  
    { registerNumber: regNo },  
    { age: 23 },  
    { new: true }  
  );  
  
  console.log('Student Updated:',  
student);  
}
```

CRUD Operations using Mongoose

contd...

// Delete

```
async function
deleteStudent(regNo) {
  const result = await
Student.deleteOne({ registerNumb
er: regNo });
  console.log('Deleted:', result);
}
```

// Call the functions here

```
async function main() {
  await createStudent();
```

```
await getStudents();
await updateStudent('REG101');
await deleteStudent('REG101');
mongoose.connection.close();
// Close the DB after operations
}
```

```
main();
```

To Run in VS Code:
node app.js